

Paper

1 Tóm tắt paper: Goodhart's Law in Reinforcement Learning

1.1 Ý tưởng trung tâm

Paper nghiên cứu hiện tượng **Goodhart's Law** trong Reinforcement Learning (RL). Goodhart's Law thường được phát biểu là:

When a measure becomes a target, it ceases to be a good measure.

Trong RL, để huấn luyện agent, ta cần định nghĩa một reward function. Tuy nhiên, với các nhiệm vụ phức tạp trong thế giới thực, rất khó để thiết kế một reward function mô tả hoàn hảo mục tiêu thật. Vì vậy, reward thường chỉ là một **proxy** cho true objective, chứ không phải chính true objective.

Nếu gọi:

$$R_0 = \text{true reward},$$

$$R_1 = \text{proxy reward},$$

thì vấn đề chính là:

$$\max_{\pi} J_{R_1}(\pi) \not\Rightarrow \max_{\pi} J_{R_0}(\pi).$$

Nói cách khác, tối ưu proxy reward không đảm bảo tối ưu true reward.

Paper chỉ ra rằng khi tối ưu một proxy reward không hoàn hảo, hiệu năng theo true reward thường có dạng:

$$\text{ban đầu tăng} \rightarrow \text{đạt cực đại} \rightarrow \text{giảm}.$$

Đây chính là hiện tượng **Goodharting**.

1.2 Motivation

Trong RL, bài toán thường được mô hình hóa bằng Markov Decision Process:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \tau, \mu, R, \gamma \rangle,$$

trong đó:

- $\mathcal{S}$: tập trạng thái,
- $\mathcal{A}$: tập hành động,
- τ : transition function,
- μ : initial-state distribution,
- R : reward function,
- γ : discount factor.

Expected return của policy π theo reward R là:

$$J_R(\pi) = \mathbb{E}_{\xi \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right].$$

Trong thực tế, reward R có thể bị misspecified. Khi đó, agent không tối ưu mục tiêu thật, mà tối ưu một proxy reward.

Điều này đặc biệt nguy hiểm trong các ứng dụng safety-critical, vì agent có thể tìm ra cách tăng reward nhưng không thực sự hoàn thành mục tiêu mong muốn.

1.3 Goodharting trong RL

Paper định nghĩa hiện tượng Goodharting như sau:

Giả sử R_0 là true reward và R_1 là proxy reward. Khi tăng optimization pressure lên R_1 , policy thu được là π_λ , với $\lambda \in [0, 1]$.

Nếu:

$$J_{R_0}(\pi_\lambda)$$

ban đầu tăng khi λ tăng, nhưng sau một ngưỡng nào đó lại giảm, thì ta có Goodharting. Một cách trực quan:

$$\lambda \uparrow \Rightarrow J_{R_1}(\pi_\lambda) \uparrow,$$

nhưng:

$J_{R_0}(\pi_\lambda)$ có thể tăng rồi giảm.

Điều này có nghĩa là càng tối ưu proxy mạnh, agent càng dễ khai thác sai lệch giữa proxy reward và true reward.

1.4 Đo mức độ Goodharting

Paper đề xuất thước đo chính là **Normalised Drop Height** (NDH).

Giả sử:

$$f(\lambda) = J_{R_0}(\pi_\lambda),$$

trong đó π_λ là policy thu được khi tối ưu proxy reward với optimization pressure λ .
Khi đó:

$$\lambda^* = \arg \max_{\lambda \in [0,1]} f(\lambda).$$

NDH được định nghĩa là độ giảm true reward sau điểm tốt nhất:

$$\text{NDH}(f) = f(\lambda^*) - f(1).$$

Nếu:

$$\text{NDH} > 0,$$

thì có Goodharting.

Trực quan:

NDH = true reward tốt nhất trong quá trình train—true reward sau khi tối ưu proxy tối đa.

NDH càng lớn thì Goodharting càng nghiêm trọng.

1.5 Kết quả thực nghiệm: Goodharting là hiện tượng phổ biến

Paper kiểm tra Goodharting trên nhiều môi trường RL khác nhau, bao gồm:

- Gridworld,
- Cliff,
- RandomMDP,
- TreeMDP.

Các tác giả thay đổi nhiều yếu tố:

- số trạng thái,
- số hành động,
- discount factor γ ,
- sparsity của reward,
- loại reward,
- loại optimization pressure.

Kết quả chính:

Goodharting xảy ra trong khoảng 19.3% thí nghiệm.

Điều này cho thấy Goodharting không phải hiện tượng hiếm hoặc chỉ xảy ra trong ví dụ nhân tạo. Nó xuất hiện trong nhiều loại môi trường và reward khác nhau.

Paper cũng quan sát rằng khi proxy reward càng xa true reward, mức độ Goodharting thường càng lớn.

1.6 Giải thích hình học của Goodhart's Law

Một đóng góp quan trọng của paper là giải thích Goodharting bằng hình học của occupancy measure.

Với một policy π , occupancy measure được định nghĩa là:

$$\eta^\pi(s, a) = \sum_{t=0}^{\infty} \gamma^t P(s_t = s, a_t = a \mid \pi).$$

Khi đó expected return có thể viết thành tích vô hướng:

$$J_R(\pi) = \eta^\pi \cdot R.$$

Tập tất cả occupancy measures khả thi tạo thành một convex polytope:

$$\Omega = \{\eta^\pi : \pi \in \Pi\}.$$

Do đó, bài toán tối ưu RL có thể được nhìn như bài toán tối ưu một hàm tuyến tính trên một convex polytope:

$$\max_{\eta \in \Omega} \eta \cdot R.$$

Nếu tối ưu proxy reward R_1 , agent di chuyển trong không gian Ω theo hướng làm tăng:

$$\eta \cdot R_1.$$

Ban đầu, nếu R_1 tương đối gần R_0 , di chuyển theo hướng R_1 cũng làm tăng true reward:

$$\eta \cdot R_0.$$

Tuy nhiên, khi trajectory chạm vào biên của convex polytope Ω , hướng tối ưu bị đổi theo hình học của biên. Khi đó, hướng tiếp tục tăng proxy reward có thể trở thành hướng làm giảm true reward.

Đó là cơ chế hình học tạo ra Goodharting:

$$\text{maximize } R_1 \Rightarrow \text{ban đầu tăng } R_0, \text{ sau đó giảm } R_0.$$

1.7 Vai trò của khoảng cách giữa true reward và proxy reward

Paper dùng **projected angle** để đo khoảng cách giữa hai reward functions:

$$\arg(R_0, R_1).$$

Nếu góc giữa R_0 và R_1 nhỏ, hai reward induce thứ tự policy tương tự nhau. Nếu góc lớn, proxy reward dễ dẫn agent sang policy tốt theo proxy nhưng xấu theo true reward.

Một kết quả quan trọng:

$$\arg(R_0, R_1) = 0$$

nếu và chỉ nếu R_0 và R_1 tạo cùng thứ tự trên mọi policy:

$$J_{R_0}(\pi) \leq J_{R_0}(\pi') \iff J_{R_1}(\pi) \leq J_{R_1}(\pi').$$

Vì vậy, vấn đề không chỉ là reward values khác nhau, mà là liệu hai reward có xếp hạng policy theo cùng một cách hay không.

1.8 Tại sao tối ưu quá mạnh lại nguy hiểm?

Trong giai đoạn đầu, proxy reward và true reward thường tương quan:

$$R_1 \uparrow \Rightarrow R_0 \uparrow.$$

Nhưng sau một mức optimization pressure đủ lớn, agent có thể tìm ra các vùng trong policy space nơi proxy reward vẫn tăng nhưng true reward giảm:

$$R_1 \uparrow \Rightarrow R_0 \downarrow.$$

Đây là bản chất của Goodhart's Law trong RL.

Nói ngắn gọn:

Tối ưu proxy vừa phải có thể hữu ích,

nhưng tối ưu proxy quá mạnh có thể phá vỡ tương quan với mục tiêu thật.

1.9 Early stopping để tránh Goodharting

Paper đề xuất một phương pháp **early stopping**.

Ý tưởng:

Nếu ta biết một bound θ cho khoảng cách giữa proxy reward và true reward:

$$\arg(R_0, R_1) \leq \theta,$$

thì có thể dừng quá trình tối ưu trước khi proxy reward bắt đầu dẫn policy vào vùng có thể làm giảm true reward.

Kết quả lý thuyết chính:

Nếu chuỗi policy π_i được sinh ra trong quá trình tối ưu proxy reward, thì nên dừng tại điểm đầu tiên thỏa:

$$\frac{J_{R_1}(\pi_{i+1}) - J_{R_1}(\pi_i)}{\|\eta^{\pi_{i+1}} - \eta^{\pi_i}\|} < \sin(\theta) \|M_\tau R_1\|.$$

Điều này nghĩa là khi lợi ích biên theo proxy reward trở nên quá nhỏ so với uncertainty về true reward, tiếp tục tối ưu có nguy cơ gây Goodharting.

1.10 Ý nghĩa của early stopping

Early stopping trong paper có tính chất:

provably avoids Goodharting

nếu bound θ là đúng.

Tuy nhiên, nó có nhược điểm:

có thể dừng quá sớm và mất một phần true reward tiềm năng.

Thực nghiệm cho thấy early stopping tránh được Goodharting trong các thí nghiệm của paper, nhưng có thể đánh đổi bằng việc mất reward, trung bình từ khoảng 10% đến 44% tùy môi trường.

Do đó, early stopping phù hợp nhất khi:

tránh failure mode quan trọng hơn tối đa hóa reward tuyệt đối.

1.11 Worst-case robust optimization

Ngoài early stopping, paper còn đề xuất một mục tiêu tối ưu robust hơn khi không chắc true reward là gì.

Nếu true reward nằm trong tập:

$$\mathcal{F}_R^\theta = \{R : \arg(R, R_1) \leq \theta\},$$

thì thay vì chỉ tối ưu proxy reward:

$$\max_{\pi} J_{R_1}(\pi),$$

ta có thể tối ưu worst-case true reward:

$$\max_{\pi} \min_{R \in \mathcal{F}_R^\theta} J_R(\pi).$$

Paper cho thấy mục tiêu này tương đương với việc tối ưu một reward có regularization hình học:

$$J_{R_1}(\pi) - \kappa \|\eta^\pi\| \sin(\arg(\eta^\pi, R_1)),$$

trong đó:

$$\kappa = \tan(\theta) \|M_\tau R_1\|.$$

Ý nghĩa trực quan:

tăng proxy reward — phạt các policy lệch khỏi hướng an toàn.

1.12 Liên hệ với reward hacking

Goodharting là một dạng của reward gaming hoặc reward hacking.

Reward hacking xảy ra khi agent tìm cách tăng reward theo cách không mong muốn. Goodharting là trường hợp đặc biệt trong đó:

proxy reward ban đầu còn hữu ích,
nhưng sau một ngưỡng tối ưu:

proxy reward bị khai thác và mất tương quan với true objective.

Vì vậy, Goodharting mô tả một failure mode tinh vi hơn so với việc reward sai ngay từ đầu.

1.13 Liên hệ với loss function trong ML

Dù paper tập trung vào RL và reward function, ý tưởng của nó liên hệ rất gần với loss function trong ML.

Trong supervised learning, ta thường tối ưu:

$$\min_{\Theta} L_{\text{train}}(\Theta).$$

Nhưng loss trên training data cũng chỉ là proxy cho mục tiêu thật, ví dụ:

generalization, robustness, fairness, human usefulness.

Vì vậy, tương tự RL:

$$\min L_{\text{proxy}} \not\Rightarrow \max \text{true objective}.$$

Goodhart's Law giúp giải thích vì sao một mô hình có thể đạt loss thấp nhưng vẫn không tốt theo mục tiêu thực tế.

1.14 Kết luận chính

Paper đưa ra các kết luận quan trọng:

- Reward function trong RL thường chỉ là proxy cho true objective.
- Khi proxy reward bị misspecified, tối ưu nó quá mạnh có thể làm giảm true reward.
- Goodharting xảy ra khá phổ biến trong nhiều môi trường RL.
- Hiện tượng này có thể được giải thích bằng hình học của occupancy measure trên convex polytope.
- Khoảng cách góc giữa true reward và proxy reward là yếu tố quan trọng quyết định mức độ rủi ro.

- Early stopping có thể tránh Goodharting nếu biết bound cho sai lệch giữa proxy và true reward.
- Có thể thiết kế objective robust theo worst-case reward để giảm rủi ro từ reward misspecification.

1.15 Thông điệp quan trọng nhất

Trong RL, reward không nên luôn được xem là mục tiêu thật, mà thường chỉ là proxy cho mục tiêu thật.

Tối ưu proxy quá mạnh có thể khiến agent giỏi tối đa hóa reward nhưng tệ hơn theo mục tiêu thật.

Goodharting là cơ chế giải thích vì sao tối ưu hóa mạnh một mục tiêu sai lệch có thể gây thất bại.

1.16 Ý tưởng trung tâm

Paper nhấn mạnh rằng phần lớn các phương pháp học máy hiện đại đều xoay quanh việc **tối ưu hóa hàm loss**. Do đó, việc lựa chọn và thiết kế một hàm loss phù hợp là yếu tố then chốt để giải quyết thành công một bài toán machine learning.

Một mô hình học máy thường được mô tả như một hàm

$$f_{\Theta} : \Phi \rightarrow \mathcal{Y},$$

trong đó $x_i \in \Phi$ là đầu vào, $y_i \in \mathcal{Y}$ là nhãn hoặc mục tiêu, và Θ là tập tham số của mô hình.

Hàm loss đo mức độ khác nhau giữa dự đoán $f_{\Theta}(x_i)$ và giá trị thật y_i :

$$L(f_{\Theta}(x_i), y_i) \in \mathbb{R}.$$

Loss tổng quát trên toàn bộ tập dữ liệu được viết là:

$$\mathcal{L}(f_{\Theta}) = \frac{1}{N} \sum_{i=1}^N L(f_{\Theta}(x_i), y_i).$$

Bài toán huấn luyện mô hình vì vậy trở thành bài toán tối ưu:

$$\min_{\Theta} \frac{1}{N} \sum_{i=1}^N L(f_{\Theta}(x_i), y_i).$$

Nếu thêm regularization để giảm overfitting, bài toán trở thành:

$$\min_{\Theta} \frac{1}{N} \sum_{i=1}^N L(f_{\Theta}(x_i), y_i) + R(\Theta).$$

Điều này cho thấy loss function không chỉ là một công thức đo lỗi, mà là cách ta **mã hóa mục tiêu học** cho mô hình.

1.17 Đóng góp chính của paper

Paper đưa ra một taxonomy tương đối toàn diện cho các hàm loss trong machine learning. Tác giả khảo sát 43 loss functions phổ biến và phân loại chúng theo hai trục chính:

- Theo loại bài toán:
 - Regression
 - Classification
 - Generative modeling
 - Ranking
 - Energy-based modeling
- Theo nguyên lý tối ưu:
 - Error-based losses
 - Probabilistic losses
 - Margin-based losses

Taxonomy này giúp người dùng hiểu loss nào phù hợp với bài toán nào, ưu điểm và hạn chế của từng loss, cũng như mối liên hệ giữa các nhóm loss.

1.18 Các tính chất toán học quan trọng của loss

Paper nhấn mạnh rằng khả năng tối ưu một hàm loss phụ thuộc mạnh vào các tính chất toán học của nó, bao gồm:

- **Continuity**: loss liên tục giúp quá trình tối ưu ổn định hơn.
- **Differentiability**: loss khả vi cho phép sử dụng gradient descent.
- **Lipschitz continuity**: giới hạn tốc độ thay đổi của hàm, có ích cho tính ổn định và robustness.
- **Convexity**: nếu loss lồi, local minimum cũng là global minimum.

- **Strict convexity:** có thể đảm bảo nghiệm tối ưu duy nhất trong một số trường hợp.

Ví dụ, gradient descent yêu cầu loss khả vi:

$$\Theta^{(t+1)} = \Theta^{(t)} - \alpha \nabla_{\Theta} \mathcal{L}(\Theta^{(t)}),$$

trong đó α là learning rate.

Nếu loss không khả vi hoặc không lồi, việc tối ưu có thể khó hơn, cần dùng subgradient, zeroth-order optimization, hoặc derivative-free optimization.

1.19 Regularization như một phần mở rộng của loss

Regularization được xem như cách mở rộng loss để kiểm soát độ phức tạp của mô hình. Dạng tổng quát là:

$$\tilde{L}(f_{\Theta}(x_i), y_i) = L(f_{\Theta}(x_i), y_i) + \lambda \rho(\Theta),$$

trong đó $\rho(\Theta)$ là hàm regularization và λ điều khiển mức độ phạt.

Hai dạng phổ biến là:

L2 regularization / Ridge

$$\tilde{L} = L + \lambda \|\Theta\|_2^2.$$

L2 làm giảm độ lớn của trọng số, giúp mô hình bớt overfit, nhưng thường không đưa trọng số về đúng bằng 0.

L1 regularization / Lasso

$$\tilde{L} = L + \lambda \|\Theta\|_1.$$

L1 có thể đưa một số trọng số về 0, nên có tác dụng như feature selection. Tuy nhiên, nó không khả vi tại 0 và có thể khó tối ưu hơn L2.

1.20 Regression losses

Trong regression, mục tiêu là dự đoán một biến liên tục y . Các loss thường dựa trên residual:

$$r_i = y_i - f_{\Theta}(x_i).$$

1.20.1 Mean Bias Error

$$L_{\text{MBE}} = \frac{1}{N} \sum_{i=1}^N (y_i - f_{\Theta}(x_i)).$$

MBE đo bias trung bình, nhưng ít dùng làm loss để train vì lỗi dương và lỗi âm có thể triệt tiêu nhau.

1.20.2 Mean Absolute Error

$$L_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |y_i - f_{\Theta}(x_i)|.$$

MAE ít nhạy với outlier hơn MSE vì phạt lỗi theo tuyến tính. Tuy nhiên, đạo hàm tại 0 không xác định và gradient không phụ thuộc vào độ lớn lỗi, có thể làm hội tụ khó hơn.

1.20.3 Mean Squared Error

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - f_{\Theta}(x_i))^2.$$

MSE rất phổ biến trong regression. Nó phạt lỗi lớn mạnh hơn do bình phương residual, nên nhạy với outlier. Về mặt xác suất, tối thiểu hóa MSE tương đương với maximum likelihood estimation nếu giả định nhiễu Gaussian.

1.20.4 Root Mean Squared Error

$$L_{\text{RMSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - f_{\Theta}(x_i))^2}.$$

RMSE có cùng đơn vị với biến mục tiêu, dễ diễn giải hơn MSE, nhưng vẫn nhạy với outlier.

1.20.5 Huber Loss

$$L_{\delta}(r) = \begin{cases} \frac{1}{2}r^2, & |r| \leq \delta, \\ \delta \left(|r| - \frac{1}{2}\delta \right), & |r| > \delta. \end{cases}$$

Huber loss kết hợp ưu điểm của MSE và MAE: với lỗi nhỏ thì giống MSE, với lỗi lớn thì giống MAE. Do đó, nó vừa dễ tối ưu hơn MAE quanh 0, vừa ít nhạy với outlier hơn MSE. Nhược điểm là cần chọn thêm hyperparameter δ .

1.20.6 Log-cosh Loss

$$L_{\log \cosh} = \frac{1}{N} \sum_{i=1}^N \log (\cosh(f_{\Theta}(x_i) - y_i)).$$

Log-cosh gần giống MSE với lỗi nhỏ và gần giống MAE với lỗi lớn, nhưng không cần chọn ngưỡng như Huber. Đổi lại, nó tốn chi phí tính toán hơn.

1.21 Classification losses

Trong classification, mô hình gán đầu vào x vào một hoặc nhiều lớp rời rạc. Paper chia classification losses thành hai nhóm chính:

- Margin-based losses
- Probabilistic losses

1.21.1 Zero-One Loss

$$L_{0/1}(f(x), y) = \begin{cases} 1, & f(x)y < 0, \\ 0, & \text{otherwise.} \end{cases}$$

Zero-One loss trực quan vì nó đếm dự đoán đúng/sai. Tuy nhiên, nó không lồi và không khả vi, nên khó tối ưu trực tiếp. Vì vậy, người ta thường dùng surrogate losses.

1.21.2 Hinge Loss

$$L_{\text{hinge}}(f(x), y) = \max(0, 1 - yf(x)).$$

Hinge loss là surrogate loss nổi tiếng, thường dùng trong Support Vector Machines. Nó phạt các điểm bị phân loại sai hoặc nằm trong margin. Tuy nhiên, nó không trơn tại $yf(x) = 1$, nên có thể khó tối ưu hơn các loss trơn.

1.21.3 Cross-Entropy / Negative Log-Likelihood

Với binary classification:

$$L_{\text{NLL}} = - \sum_{i=1}^N [y_i \log f_{\Theta}(x_i) + (1 - y_i) \log(1 - f_{\Theta}(x_i))].$$

Cross-entropy có thể được hiểu là negative log-likelihood. Nó không chỉ quan tâm mô hình dự đoán đúng hay sai, mà còn quan tâm độ tự tin của dự đoán. Một dự đoán sai với xác suất cao sẽ bị phạt rất nặng.

Với multi-class classification, thường dùng softmax:

$$\hat{p}_k(x) = \frac{e^{s_k}}{\sum_{j=1}^K e^{s_j}},$$

và categorical cross-entropy:

$$L_{\text{CCE}} = - \sum_{k=1}^K y_k \log \hat{p}_k(x).$$

1.21.4 Kullback-Leibler Divergence

$$D_{\text{KL}}(q||p) = \int q(x) \log \frac{q(x)}{p(x)} dx.$$

KL divergence đo sự khác biệt giữa hai phân phối xác suất. Trong classification, tối thiểu hóa KL divergence thường tương đương với tối thiểu hóa cross-entropy, vì entropy của phân phối thật không phụ thuộc vào tham số mô hình.

1.21.5 Focal Loss

$$L_{\text{focal}} = -(1 - p_t)^\gamma \log(p_t).$$

Focal loss được thiết kế để xử lý class imbalance. Nó giảm trọng số của các example dễ và tập trung nhiều hơn vào các example khó hoặc bị phân loại sai. Khi $\gamma = 0$, focal loss trở về cross-entropy.

1.21.6 Dice Loss

$$L_{\text{Dice}} = 1 - \frac{2 \sum_i p_i y_i}{\sum_i p_i + \sum_i y_i}.$$

Dice loss thường dùng trong image segmentation, đặc biệt khi dữ liệu mất cân bằng giữa foreground và background.

1.21.7 Tversky Loss

$$L_{\text{Tversky}} = 1 - \frac{\sum_i p_i y_i}{\sum_i p_i y_i + \alpha \sum_i p_i (1 - y_i) + \beta \sum_i (1 - p_i) y_i}.$$

Tversky loss mở rộng Dice loss bằng cách cho phép điều chỉnh trade-off giữa false positives và false negatives thông qua α và β .

1.22 Generative losses

Trong generative modeling, mục tiêu là học phân phối dữ liệu và sinh ra mẫu mới giống dữ liệu thật.

1.22.1 VAE Loss

VAE loss dựa trên Evidence Lower Bound, gồm hai thành phần:

$$L_{\text{VAE}} = -\mathbb{E}_{q(z|x)}[\log p_\Theta(x|z)] + D_{\text{KL}}(q(z|x)||p(z)).$$

Thành phần đầu là reconstruction loss, giúp mô hình tái tạo dữ liệu. Thành phần thứ hai là KL divergence, giúp latent space có cấu trúc và gần với prior.

Nếu chỉ dùng reconstruction loss, mô hình có thể tái tạo tốt nhưng latent space không có cấu trúc. Nếu KL term quá mạnh, mô hình có thể bị posterior collapse.

1.22.2 Beta-VAE

$$L_{\beta\text{-VAE}} = L_{\text{recon}} + \beta D_{\text{KL}}.$$

Beta-VAE thêm hệ số β để điều chỉnh trade-off giữa reconstruction quality và latent disentanglement.

1.22.3 GAN Minimax Loss

$$L_{\text{minimax}}(D, G) = \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))].$$

GAN gồm generator và discriminator. Generator cố gắng tạo dữ liệu giả để đánh lừa discriminator, trong khi discriminator học cách phân biệt dữ liệu thật và giả. Loss của GAN khó tối ưu vì đây là trò chơi minimax giữa hai mô hình.

1.22.4 Wasserstein GAN Loss

Wasserstein loss thay thế cách phân biệt xác suất truyền thống bằng critic score. Nó giúp training ổn định hơn và cung cấp gradient hữu ích hơn trong nhiều trường hợp.

$$L_{\text{critic}} = D(x) - D(G(z)),$$

$$L_{\text{generator}} = D(G(z)).$$

Ưu điểm chính của Wasserstein loss là ổn định hơn minimax loss, ít nhạy hơn với kiến trúc và hyperparameter.

1.22.5 Diffusion Loss

Diffusion models học cách thêm noise vào dữ liệu rồi đảo ngược quá trình này. Một loss đơn giản thường dùng trong DDPM là:

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \left[\left\| \epsilon - \epsilon_{\Theta} \left(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2 \right].$$

Ở đây, mô hình học dự đoán noise đã được thêm vào dữ liệu. Loss này có dạng MSE, dễ tính toán và thường ổn định khi huấn luyện.

1.23 Loss functions trong LLMs

Paper cũng thảo luận loss dùng trong Transformer và Large Language Models.

1.23.1 Autoregressive Language Modeling Loss

Với mô hình kiểu GPT, nhiệm vụ là dự đoán token tiếp theo:

$$L_{AR} = - \sum_{t=1}^T \log p_{\Theta}(x_t | x_{<t}).$$

Loss này là cross-entropy trên chuỗi token. Nó khiến mô hình học phân phối xác suất của token tiếp theo dựa trên ngữ cảnh trước đó.

1.23.2 Masked Language Modeling Loss

Với mô hình kiểu BERT, một số token bị mask và mô hình phải dự đoán lại chúng:

$$L_{MLM} = - \sum_{m \in \text{Masked}} \log p_{\Theta}(x_m | x_{\setminus m}).$$

MLM loss cho phép mô hình học ngữ cảnh hai chiều, vì nó dùng cả token bên trái và bên phải.

1.23.3 Label Smoothing

Label smoothing tránh việc mô hình quá tự tin bằng cách phân phối một phần nhỏ xác suất cho các class sai:

$$\tilde{y}_i = (1 - \epsilon)y_i + \frac{\epsilon}{K}.$$

Loss khi đó là:

$$L_{\text{label smoothing}} = - \sum_{i=1}^K \tilde{y}_i \log p_i.$$

1.23.4 Knowledge Distillation Loss

Trong knowledge distillation, một mô hình nhỏ học từ mô hình lớn:

$$L_{KD} = (1 - \alpha)L_{CE}(p_{\text{student}}, y) + \alpha D_{KL}(p_{\text{student}}, p_{\text{teacher}}).$$

Loss này kết hợp nhân thật với phân phối đầu ra của teacher model.

1.24 Ranking losses

Ranking losses không dự đoán trực tiếp nhãn hoặc giá trị, mà học quan hệ thứ tự hoặc khoảng cách giữa các đối tượng.

1.24.1 Pairwise Ranking Loss

$$L_{\text{pairwise}} = \begin{cases} d(r_a, r_p), & \text{positive pair,} \\ \max(0, m - d(r_a, r_n)), & \text{negative pair.} \end{cases}$$

Loss này kéo các cặp giống nhau lại gần và đẩy các cặp khác nhau ra xa ít nhất một margin m .

1.24.2 Triplet Loss

$$L_{\text{triplet}} = \max(0, m + d(r_a, r_p) - d(r_a, r_n)).$$

Triplet loss dùng ba mẫu: anchor, positive và negative. Mục tiêu là embedding của anchor gần positive hơn negative ít nhất một margin.

1.24.3 Contrastive Ranking Loss / NT-Xent

$$L_{\text{NT-Xent}} = -\log \frac{\exp(\text{sim}(x_a, x_p)/\tau)}{\sum_{i=1}^N \exp(\text{sim}(x_a, x_n^i)/\tau)}.$$

NT-Xent phổ biến trong contrastive learning. Nó tăng similarity giữa positive pair và giảm similarity với nhiều negative samples.

1.25 Energy-based losses

Energy-Based Models gán một giá trị năng lượng cho mỗi cặp (x, y) :

$$E_{\Theta}(x, y).$$

Dự đoán được chọn bằng cách tìm output có năng lượng thấp nhất:

$$\hat{y} = \arg \min_y E_{\Theta}(x, y).$$

Ý tưởng huấn luyện là:

correct outputs $\rightarrow$ low energy,

incorrect outputs $\rightarrow$ high energy.

Một dạng tổng quát của loss trong EBM là:

$$L_{\text{EBM}} = \frac{1}{N} \sum_{i=1}^N L_{\text{EBM}}(y_i, E_{\Theta}(x_i, Y)) + \text{regularizer}.$$

1.25.1 Energy Loss

$$L_{\text{energy}} = E_{\Theta}(x_i, y_i).$$

Loss này đơn giản vì chỉ kéo năng lượng của output đúng xuống thấp. Tuy nhiên, nó không trực tiếp đẩy năng lượng của output sai lên cao, nên có nguy cơ energy collapse.

1.25.2 Generalized Perceptron Loss

$$L_{\text{perceptron}} = E_{\Theta}(x_i, y_i) - \min_y E_{\Theta}(x_i, y).$$

Loss này so sánh năng lượng của output đúng với output có năng lượng thấp nhất. Tuy nhiên, nó không đảm bảo margin rõ ràng giữa output đúng và output sai.

1.25.3 Energy-based Hinge Loss

$$L_{\text{hinge}} = \max(0, m + E_{\Theta}(x_i, y_i) - E_{\Theta}(x_i, \bar{y}_i)),$$

trong đó $\bar{y}_i$ là output sai có năng lượng thấp nhất.

Loss này ép năng lượng của output đúng phải thấp hơn output sai ít nhất một margin m .

1.26 Kết luận chính

Paper kết luận rằng lựa chọn loss function là một quyết định nền tảng trong machine learning. Một loss tốt phải phù hợp với:

- Loại bài toán: regression, classification, generative modeling, ranking, hoặc energy-based modeling.
- Tính chất dữ liệu: có outlier hay không, mất cân bằng lớp hay không, nhãn có nhiều hay không.
- Mục tiêu thực tế: cần tối ưu sai số tuyệt đối, xác suất, margin, ranking, hay chất lượng sinh mẫu.
- Điều kiện tối ưu hóa: loss có khả vi, lỗi, ổn định gradient hay không.
- Khả năng generalization: có cần regularization để tránh overfitting hay không.

Nói cách khác, loss function là cầu nối giữa mục tiêu thực tế của con người và quá trình tối ưu hóa của mô hình. Nếu loss được chọn sai, mô hình có thể tối ưu rất tốt về mặt toán học nhưng lại không giải quyết đúng bài toán thực tế.

1.27 Thông điệp quan trọng nhất

Mô hình học cái mà loss yêu cầu nó tối ưu, không nhất thiết học đúng điều ta mong muốn.

Vì vậy, giải thích và lựa chọn loss function trở thành một vấn đề trung tâm của machine learning.